Lab No: 05

Student Name: Hania Nawab

Roll No: 17

Task 1:	

Code:	#include <iostream>
	#include <string>
	using namespace std;
	
	struct node{
	 int TaskID;
	 string TaskDescription;
	 bool status;
	 node *next;
	
	};
	
	node *head = NULL;
	
	bool idExists(int id)
	{
	  node *tmp = head;
	  while(tmp!=NULL)
	  {
	  if(tmp->TaskID == id)
	  {
	  return true;
	  }
	  tmp = tmp->next;
	  }
	  return false;
	}

```
void insertAtStart(int value,string descript)
{
    if (idExists(value))
    {
        cout<<"Task ID already exists!"<<endl;
        return;
    }
    node *temp =new node();
    temp->TaskID = value;
    temp->TaskDescription = descript;
    temp->status = false;
    temp->next = head;
    head = temp;
    return;
}

void insertAfterCertainPosition(int value,int id,string descript)
{
   if (idExists(value))
   {
       cout<<"Task ID already exists!"<<endl;
       return;
   }

   node *tmp = head;

  while(tmp!=NULL && tmp->TaskID != id)
  {
     tmp = tmp->next;
  }

  if (tmp == NULL)
  {
```

```
      cout<<"Task ID "<<id<<" not found."<<endl;
      return;
   }

    node *temp =new node();
    temp->TaskID = value;
    temp->TaskDescription = descript;
    temp->status = false;
    temp->next = tmp->next;
    tmp->next = temp;
}

void insertAtEnd(int value,string descript)
{
   if (idExists(value))
   {
      cout<<"Task ID already exists!"<<endl;
      return;
   }
   node *temp = new node();
   temp->TaskID = value;
   temp->TaskDescription = descript;
   temp->status = false;
   temp->next = NULL;

   if (head == NULL)
   {
      head = temp;

   }else{

     node *tem = head;
      while(tem->next != NULL)
      {
```

```
            tem = tem->next;
        }
        tem->next = temp;
    }
}

void DeleteAtAnyPosition(int id)
{
    node *tmp = head;
    int pos = 1;

   while(tmp!=NULL)
   {
      if(tmp->TaskID == id)
    {
      break;
    }
    tmp = tmp->next;
    pos++;
   }

    if (tmp == NULL)
    {
       cout<<"Task ID "<<id<<" not found."<<endl;
       return;
    }

    if (pos == 1)
    {
      node *Delet = head;
     head = head->next;
     delete Delet;
     return;
    }
```

```
 else{
    node *prev = head;
    for(int i=0; i<pos-2; i++)
    {
       prev = prev->next;
    }
    node *Delet = prev->next;
    prev->next = Delet->next;
    delete Delet;
    return;
 }
}

void SearchValue(int value)
{
    node *temp = head;
    int pos = 1;

   while(temp!=NULL)
   {
       if(temp->TaskID == value)
    {
       cout<<value<< " Exist at Positon : "<< pos<<endl;
       cout<<temp->TaskID <<endl;
       cout<<temp->TaskDescription<<endl;

       if(temp->status == true)
       {
          cout<<"Status : Completed! "<<endl;
       }
       if(temp->status == false)
       {
          cout<<"Status : Pending... "<<endl;
```

```
      }
      return;
    }
    temp = temp->next;
    pos++;
   }
   cout<<value<<" does not exist. "<<endl;
}

void showPendingTask()
{
    node *temp = head;
    bool found = false;
    if (temp == NULL)
    {
        cout << "List is empty!" << endl;
        return;
    }
    while(temp != NULL)
    {
        if(temp->status == false)
        {
          cout<<(temp->TaskID)<<endl;
          cout<<temp->TaskDescription<<endl;
          cout<<"Status : Pending... "<<endl;
          found = true;
        }
        temp = temp->next;
    }
    if (found == false)
    {
        cout<<"No Task is Pending."<<endl;
    }
}
```

```
void CountNoOfTasks()
{
   int Count =0;
   node *temp = head;
   if (temp == NULL)
   {
      cout << "List is empty!" << endl << "Count = 0 "<<endl;
      return;
   }
   while(temp != NULL)
   {
      Count++;
      temp = temp->next;
   }
   cout<<"The Number of Task = "<<Count<<endl;
}

void MarkTaskComplete(int id)
{
   node *tmp = head;

  while(tmp!=NULL)
  {
     if(tmp->TaskID == id)
   {
     break;
   }
   tmp = tmp->next;
  }

   if (tmp == NULL)
   {
      cout<<"Task ID "<<id<<" not found."<<endl;
```

```
        return;
    }

    tmp->status = true;
}

void showAllTask()
{
    node *temp = head;
    if (temp == NULL)
    {
        cout << "List is empty!" << endl;
        return;
    }

    cout<<"---Pending Task---"<<endl;
    while(temp != NULL)
    {
        if(temp->status == false)
        {
          cout<<(temp->TaskID)<<endl;
          cout<<temp->TaskDescription<<endl;
          cout<<"Status : Pending... "<<endl;
        }
        temp = temp->next;
    }

    temp = head;
    cout<<"---Completed Task---"<<endl;
    while(temp != NULL)
    {
        if(temp->status == true)
        {
          cout<<(temp->TaskID)<<endl;
```

```
        cout<<temp->TaskDescription<<endl;
        cout<<"Status : Completed! "<<endl;
      }
      temp = temp->next;
   }
}


int main()
{
   int x,pos,value;
   string descript;

   do{
   cout << " Single Linked List " << endl;
   cout << "1.Add a new task at the beginning." <<endl;
   cout << "2.Insert a task after a given Task ID." <<endl;
   cout << "3.Add a task at the end."<<endl;
   cout << "4.Delete a task using Task ID."<<endl;
   cout << "5.Search for a task."<<endl;
   cout << "6.Display all pending tasks." <<endl;
   cout << "7.Count the number of tasks." <<endl;
   cout << "8. Mark a task as completed." <<endl;
   cout << "9.Display completed and pending tasks separately." <<endl;
   cout << "0. Exit "<<endl;
   cin >> x;
   cout<<endl;

   if (cin.fail())
   {
      cin.clear();
      cin.ignore(1000, '\n');
      x = -1;
   }
```

```
if(x<0 || x>9)
{
   cout<< "Invalid input! TRY AGAIN "<<endl;
}
if (x == 1) {
        cout << "Enter TaskID to insert at first: ";
        cin >> value;
        cin.ignore();
        cout << "Enter Task Description: ";
        getline(cin, descript);
        insertAtStart(value, descript);
    }
    else if (x == 2) {
        cout << "After which TaskID you want to enter the new task: ";
        cin >> pos;
        cout << "Enter TaskID you want to add: ";
        cin >> value;
        cin.ignore();
        cout << "Enter Task Description: ";
        getline(cin, descript);
        insertAfterCertainPosition(value, pos, descript);
    }
    else if (x == 3) {
        cout << "Enter TaskID to insert at end: ";
        cin >> value;
        cin.ignore();
        cout << "Enter Task Description: ";
        getline(cin, descript);
        insertAtEnd(value, descript);
    }
    else if (x == 4) {
        cout << "Enter TaskID which you want to delete: ";
        cin >> pos;
```

```
            DeleteAtAnyPosition(pos);
        }
        else if (x == 5) {
            cout << "Enter TaskID you want to Search: ";
            cin >> value;
            SearchValue(value);
        }
        else if (x == 6) {
            showPendingTask();
        }
        else if (x == 7) {
            CountNoOfTasks();
        }
        else if (x == 8) {
            cout << "Enter Task Id you want to mark as completed: ";
            cin >> value;
            MarkTaskComplete(value);
        }
        else if (x == 9) {
            showAllTask();
        }
    } while (x != 0);

    while(head != NULL)
    {
        node *Delet = head;
        head = head->next;
        delete Delet;
    }

    cout << "Program Ends! Thank you\n";
    return 0;
}
```

Screenshots:

1. Add a new task at the beginning:

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
1

Enter TaskID to insert at first: 1
Enter Task Description: ADDING DATA
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
1

Enter TaskID to insert at first: 2
Enter Task Description: MAKING SOMETHING
```

2. Insert a task after a given Task ID:

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
2

After which TaskID you want to enter the new task: 2
Enter TaskID you want to add: 3
```

3.Add a task at the end:

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
3

Enter TaskID to insert at end: 5
Enter Task Description: TESTING
```

4.Delete a task using Task ID

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
4

Enter TaskID which you want to delete: 2
```

5.Search for a task

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
5

Enter TaskID you want to Search: 4
4 does not exist.
```

6.Display all pending tasks.

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
6

0
ADDING TASK
Status : Pending...
3
UPDATING
Status : Pending...
```

7.Count the number of tasks.

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
7

The Number of Task = 3
```

8. Mark a task as completed.

```
Enter Task Id you want to mark as completed: 5
Task ID 5 not found.
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
8

Enter Task Id you want to mark as completed: 1
```

9.Display completed and pending tasks separately

```
 Single Linked List
1.Add a new task at the beginning.
2.Insert a task after a given Task ID.
3.Add a task at the end.
4.Delete a task using Task ID.
5.Search for a task.
6.Display all pending tasks.
7.Count the number of tasks.
8. Mark a task as completed.
9.Display completed and pending tasks separately.
0. Exit
9

---Pending Task---
0
ADDING TASK
Status : Pending...
3
UPDATING
Status : Pending...
---Completed Task---
1
ADDING TASK
Status : Completed!
```

	0.Exit Single Linked List 1.Add a new task at the beginning. 2.Insert a task after a given Task ID. 3.Add a task at the end. 4.Delete a task using Task ID. 5.Search for a task. 6.Display all pending tasks. 7.Count the number of tasks. 8. Mark a task as completed. 9.Display completed and pending tasks separately. 0. Exit 0 Program Ends! Thank you

Task 2:	

Code:	#include <iostream>
	#include <string>
	using namespace std;
	
	struct node{
	string URL;
	string Title;
	node *prev;
	node *next;
	
	};
	
	node *head = NULL;
	node *tail = NULL;
	node *current = NULL;
	
	void visitPage(string url,string title)
	{
	if (current != NULL)
	{
	node *tmp = current->next;
	while(tmp != NULL)
	{
	node *Delet = tmp;
	tmp = tmp->next;
	delete Delet;
	}
	current->next = NULL;
	tail = current;
	}
	
	node *temp = new node();
	temp->URL = url;

```
    temp->Title = title;
    temp->next = NULL;
    temp->prev = tail;

    if (tail == NULL)
    {
        head = temp;
    }
    else
    {
        tail->next = temp;
    }
    tail = temp;
    current = temp;
    cout<<"Visited : "<<title<<" ( "<<url<<" )"<<endl;
}

void goBack()
{
    if (current == NULL)
    {
        cout << "History is empty!" << endl;
        return;
    }
    if (current->prev == NULL)
    {
        cout << "No earlier page. Already at the oldest page." << endl;
        return;
    }
    current = current->prev;
    cout<<"Back to : "<<current->Title<<" ( "<<current->URL<<" )"<<endl;
}

void goForward()
{
    if (current == NULL)
    {
        cout << "History is empty!" << endl;
        return;
    }
    if (current->next == NULL)
    {
        cout << "No forward page. Already at the newest page." << endl;
        return;
    }
    current = current->next;
    cout<<"Forward to : "<<current->Title<<" ( "<<current->URL<<" )"<<endl;
}

void displayHistory()
{
    node *temp = head;
    int pos = 1;
    if (temp == NULL)
    {
        cout << "History is empty!" << endl;
```

```
        return;
    }
    cout<<"---History (oldest to newest)---"<<endl;
    while(temp != NULL)
    {
        cout<<pos<<". "<<temp->Title<<" | "<<temp->URL;
        if (temp == current)
        {
            cout<<"  <-- current page";
        }
        cout<<endl;
        temp = temp->next;
        pos++;
    }
}

void displayReverse()
{
    node *temp = tail;
    int pos = 1;
    if (temp == NULL)
    {
        cout << "History is empty!" << endl;
        return;
    }
    cout<<"---History (newest to oldest)---"<<endl;
    while(temp != NULL)
    {
        cout<<pos<<". "<<temp->Title<<" | "<<temp->URL;
        if (temp == current)
        {
            cout<<"  <-- current page";
        }
        cout<<endl;
        temp = temp->prev;
        pos++;
    }
}

void DeletePage(string url)
{
    node *tmp = head;

   while(tmp!=NULL)
   {
      if(tmp->URL == url)
    {
       break;
    }
    tmp = tmp->next;
   }

    if (tmp == NULL)
    {
        cout<<url<<" not found. Nothing deleted."<<endl;
        return;
```

```
    }

    if (tmp->prev != NULL)
    {
        tmp->prev->next = tmp->next;
    }
    else
    {
        head = tmp->next;
    }

    if (tmp->next != NULL)
    {
        tmp->next->prev = tmp->prev;
    }
    else
    {
        tail = tmp->prev;
    }

    if (current == tmp)
    {
        if (tmp->prev != NULL)
        {
            current = tmp->prev;
        }
        else
        {
            current = tmp->next;
        }
    }

    delete tmp;
    cout<<url<<" deleted from history."<<endl;
}

void SearchURL(string url)
{
    node *temp = head;
    int pos = 1;

   while(temp!=NULL)
    {
        if(temp->URL == url)
    {
        cout<<url<< " Exist at Positon : "<< pos<<endl;
        cout<<temp->Title<<endl;
        return;
    }
    temp = temp->next;
    pos++;
    }
   cout<<url<<" does not exist. "<<endl;
}

void clearHistory()
```

```
{
   while(head != NULL)
   {
      node *Delet = head;
      head = head->next;
      delete Delet;
   }
   tail = NULL;
   current = NULL;
   cout<<"History cleared."<<endl;
}


int main()
{
   int x;
   string url,title;

   do{
   cout << " Browser History " << endl;
   cout << "1.Visit a new page." <<endl;
   cout << "2.Go back." <<endl;
   cout << "3.Go forward."<<endl;
   cout << "4.Display history (oldest to newest)."<<endl;
   cout << "5.Delete a page using URL."<<endl;
   cout << "6.Search for a URL." <<endl;
   cout << "7.Display history (newest to oldest)." <<endl;
   cout << "8.Clear history." <<endl;
   cout << "0. Exit "<<endl;
   cin >> x;
   cout<<endl;

   if (cin.fail())
   {
      cin.clear();
      cin.ignore(1000, '\n');
      x = -1;
   }

   if(x<0 || x>8)
   {
      cout<< "Invalid input! TRY AGAIN "<<endl;
   }
   if (x == 1) {
         cout << "Enter URL: ";
         cin >> url;
         cin.ignore();
         cout << "Enter Page Title: ";
         getline(cin, title);
         visitPage(url, title);
      }
      else if (x == 2) {
         goBack();
      }
      else if (x == 3) {
         goForward();
```

```
    }
    else if (x == 4) {
        displayHistory();
    }
    else if (x == 5) {
        cout << "Enter URL which you want to delete: ";
        cin >> url;
        DeletePage(url);
    }
    else if (x == 6) {
        cout << "Enter URL you want to Search: ";
        cin >> url;
        SearchURL(url);
    }
    else if (x == 7) {
        displayReverse();
    }
    else if (x == 8) {
        clearHistory();
    }
  } while (x != 0);

  while(head != NULL)
  {
    node *Delet = head;
    head = head->next;
    delete Delet;
  }

  cout << "Program Ends! Thank you\n";
  return 0;
}
```

Screenshot:

1.Visit a new page

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
1

Enter URL: namal@.edu.pk
Enter Page Title: namal
Visited : namal ( namal@.edu.pk )
```

2.Go back

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
2

No earlier page. Already at the oldest page.
```

3.Go forward

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
3

No forward page. Already at the newest page.
```

4.Display history (oldest to newest)

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
4

---History (oldest to newest)---
1. namal | namal@.edu.pk  <-- current page
```

5.Delete a page using URL

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
5

Enter URL which you want to delete: mi@.
mi@. not found. Nothing deleted.
```

6.Search for a URL

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
6

Enter URL you want to Search: namal@edu.pk
namal@edu.pk Exist at Positon : 1
NAMAL
```

7.Display history (newest to oldest).

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
7

---History (newest to oldest)---
1. NAMAL | namal@edu.pk  <-- current page
```

8.Clear history

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
8

History cleared.
```

0.Exit

```
 Browser History
1.Visit a new page.
2.Go back.
3.Go forward.
4.Display history (oldest to newest).
5.Delete a page using URL.
6.Search for a URL.
7.Display history (newest to oldest).
8.Clear history.
0. Exit
0

Program Ends! Thank you
```

Task 3:

Code:

```
#include <iostream>
#include <string>
using namespace std;

struct node{
 int LectureID;
 string Title;
 int Duration;
 node *next;

};

node *last = NULL;
node *current = NULL;
bool repeatMode = false;

void playLecture(node *l)
```

```
{
   cout<<"Now playing : [ "<<l->LectureID<<" ] "<<l->Title<<" ( "<<l->Duration<<" min
)"<<endl;
}

void addLecture(int value,string title,int duration)
{
   if (last != NULL)
   {
      node *tmp = last->next;
      do{
         if(tmp->LectureID == value)
         {
            cout<<"Lecture ID "<<value<<" already exists."<<endl;
            return;
         }
         tmp = tmp->next;
      }while(tmp != last->next);
   }

   node *temp = new node();
   temp->LectureID = value;
   temp->Title = title;
   temp->Duration = duration;

   if (last == NULL)
   {
      temp->next = temp;
      last = temp;
   }
   else
   {
      temp->next = last->next;
      last->next = temp;
      last = temp;
   }
   cout<<"Lecture added : "<<title<<endl;
}

void DeleteLecture(int id)
{
   if (last == NULL)
   {
      cout << "Playlist is empty!" << endl;
      return;
   }

   node *prev = last;
   node *tmp = last->next;
   bool found = false;

   do{
      if(tmp->LectureID == id)
      {
         found = true;
         break;
```

```
        }
        prev = tmp;
        tmp = tmp->next;
    }while(prev != last);

    if (found == false)
    {
        cout<<"Lecture ID "<<id<<" not found."<<endl;
        return;
    }

    if (tmp == tmp->next)
    {
        last = NULL;
        current = NULL;
    }
    else
    {
        prev->next = tmp->next;
        if (tmp == last)
        {
            last = prev;
        }
        if (current == tmp)
        {
            current = tmp->next;
        }
    }
    cout<<"Lecture "<<id<<" ( "<<tmp->Title<<" ) deleted."<<endl;
    delete tmp;
}

void startPlaylist()
{
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl;
        return;
    }
    current = last->next;
    playLecture(current);
}

void playNext()
{
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl;
        return;
    }
    if (current == NULL)
    {
        startPlaylist();
        return;
    }
    if (current == last && repeatMode == false)
```

```
    {
        cout << "End of playlist. Turn on repeat mode to loop." << endl;
        return;
    }
    current = current->next;
    playLecture(current);
}

void showAllLectures()
{
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl;
        return;
    }

    cout<<"---Playlist ( Repeat : ";
    if (repeatMode == true)
    {
        cout<<"ON )---"<<endl;
    }
    else
    {
        cout<<"OFF )---"<<endl;
    }

    node *temp = last->next;
    do{
        cout<<temp->LectureID<<". "<<temp->Title<<" - "<<temp->Duration<<" min";
        if (temp == current)
        {
            cout<<"  <-- playing";
        }
        cout<<endl;
        temp = temp->next;
    }while(temp != last->next);
}

void SearchLecture(int value)
{
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl;
        return;
    }

    node *temp = last->next;
    int pos = 1;

    do{
        if(temp->LectureID == value)
        {
            cout<<value<< " Exist at Positon : "<< pos<<endl;
            cout<<temp->Title<<endl;
            cout<<temp->Duration<<" min"<<endl;
            return;
```

```
        }
        temp = temp->next;
        pos++;
    }while(temp != last->next);

    cout<<value<<" does not exist. "<<endl;
}

void CountLectures()
{
    int Count =0;
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl << "Count = 0 "<<endl;
        return;
    }

    node *temp = last->next;
    do{
        Count++;
        temp = temp->next;
    }while(temp != last->next);

    cout<<"The Number of Lectures = "<<Count<<endl;
}

void skipLecture()
{
    if (current == NULL)
    {
        cout << "No lecture is playing. Start the playlist first." << endl;
        return;
    }
    cout<<"Skipped : "<<current->Title<<endl;
    playNext();
}

void autoRepeat(int n)
{
    if (last == NULL)
    {
        cout << "Playlist is empty!" << endl;
        return;
    }
    repeatMode = true;
    cout<<"Repeat mode ON. Auto-playing "<<n<<" lectures..."<<endl;
    for(int i=0; i<n; i++)
    {
        playNext();
    }
}


int main()
{
    int x,value,dur;
```

```
string title;

do{
cout << " Smart Learning Playlist " << endl;
cout << "1.Add a lecture at the end." <<endl;
cout << "2.Delete a lecture using Lecture ID." <<endl;
cout << "3.Start the playlist."<<endl;
cout << "4.Play the next lecture."<<endl;
cout << "5.Display all lectures."<<endl;
cout << "6.Search for a lecture." <<endl;
cout << "7.Count the number of lectures." <<endl;
cout << "8.Skip the current lecture." <<endl;
cout << "9.Turn repeat mode on/off." <<endl;
cout << "10.Auto-repeat the playlist." <<endl;
cout << "0. Exit "<<endl;
cin >> x;
cout<<endl;

if (cin.fail())
{
   cin.clear();
   cin.ignore(1000, '\n');
   x = -1;
}

if(x<0 || x>10)
{
   cout<< "Invalid input! TRY AGAIN "<<endl;
}
if (x == 1) {
       cout << "Enter LectureID: ";
       cin >> value;
       cin.ignore();
       cout << "Enter Lecture Title: ";
       getline(cin, title);
       cout << "Enter Duration (min): ";
       cin >> dur;
       addLecture(value, title, dur);
   }
   else if (x == 2) {
       cout << "Enter LectureID which you want to delete: ";
       cin >> value;
       DeleteLecture(value);
   }
   else if (x == 3) {
       startPlaylist();
   }
   else if (x == 4) {
       playNext();
   }
   else if (x == 5) {
       showAllLectures();
   }
   else if (x == 6) {
       cout << "Enter LectureID you want to Search: ";
       cin >> value;
```

```
            SearchLecture(value);
        }
        else if (x == 7) {
            CountLectures();
        }
        else if (x == 8) {
            skipLecture();
        }
        else if (x == 9) {
            repeatMode = !repeatMode;
            if (repeatMode == true)
            {
                cout << "Repeat mode is now ON" << endl;
            }
            else
            {
                cout << "Repeat mode is now OFF" << endl;
            }
        }
        else if (x == 10) {
            cout << "How many lectures to auto-play? ";
            cin >> value;
            autoRepeat(value);
        }
    } while (x != 0);

    if (last != NULL)
    {
        node *tem = last->next;
        last->next = NULL;
        while(tem != NULL)
        {
            node *Delet = tem;
            tem = tem->next;
            delete Delet;
        }
    }

    cout << "Program Ends! Thank you\n";
    return 0;
}
```

Screenshot:	1.Add a lecture at the end

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
1

Enter LectureID: 1
Enter Lecture Title: DS
Enter Duration (min): 40
Lecture added : DS
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
1

Enter LectureID: 2
Enter Lecture Title: SE
Enter Duration (min): 120
Lecture added : SE
```

2.Delete a lecture using Lecture ID

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
2

Enter LectureID which you want to delete: 2
Lecture 2 ( SE ) deleted.
```

3.Start the playlist

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
3

Now playing : [ 1 ] DS ( 40 min )
```

4.Play the next lecture

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
4

End of playlist. Turn on repeat mode to loop.
```

5.Display all lectures

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
5

---Playlist ( Repeat : OFF )---
1. DS - 40 min  <-- playing
```

6.Search for a lecture

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
6

Enter LectureID you want to Search: 3
3 does not exist.
```

7.Count the number of lectures

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
7

The Number of Lectures = 1
```

8.Skip the current lecture

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
8

Skipped : DS
End of playlist. Turn on repeat mode to loop.
```

9.Turn repeat mode on/off

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
9

Repeat mode is now ON
```

10.Auto-repeat the playlist

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
10

How many lectures to auto-play? 3
Repeat mode ON. Auto-playing 3 lectures.
Now playing : [ 1 ] DS ( 40 min )
Now playing : [ 1 ] DS ( 40 min )
Now playing : [ 1 ] DS ( 40 min )
```

0.Exit

```
 Smart Learning Playlist
1.Add a lecture at the end.
2.Delete a lecture using Lecture ID.
3.Start the playlist.
4.Play the next lecture.
5.Display all lectures.
6.Search for a lecture.
7.Count the number of lectures.
8.Skip the current lecture.
9.Turn repeat mode on/off.
10.Auto-repeat the playlist.
0. Exit
0

Program Ends! Thank you
```

Task 4:

Code:

```
#include <iostream>
#include <string>
using namespace std;

const int QUANTUM = 3;

struct node{
 int RequestID;
```

```
 int UserID;
 string ModelName;
 int RemainingTime;
 node *next;

};

node *last = NULL;
node *current = NULL;

void addRequest(int value,int uid,string model,int time)
{
   if (last != NULL)
   {
      node *tmp = last->next;
      do{
         if(tmp->RequestID == value)
         {
            cout<<"Request "<<value<<" already exists."<<endl;
            return;
         }
         tmp = tmp->next;
      }while(tmp != last->next);
   }

   node *temp = new node();
   temp->RequestID = value;
   temp->UserID = uid;
   temp->ModelName = model;
   temp->RemainingTime = time;

   if (last == NULL)
   {
      temp->next = temp;
      last = temp;
      current = temp;
   }
   else
   {
      temp->next = last->next;
      last->next = temp;
      last = temp;
   }
   cout<<"Request "<<value<<" added ( User "<<uid<<", "<<model<<", "<<time<<" units
)."<<endl;
}

bool DeleteRequest(int id)
{
   if (last == NULL)
   {
      cout << "No active requests!" << endl;
      return false;
   }

   node *prev = last;
```

```
    node *tmp = last->next;
    bool found = false;

    do{
        if(tmp->RequestID == id)
        {
            found = true;
            break;
        }
        prev = tmp;
        tmp = tmp->next;
    }while(prev != last);

    if (found == false)
    {
        cout<<"Request "<<id<<" not found."<<endl;
        return false;
    }

    if (tmp == tmp->next)
    {
        last = NULL;
        current = NULL;
    }
    else
    {
        prev->next = tmp->next;
        if (tmp == last)
        {
            last = prev;
        }
        if (current == tmp)
        {
            current = tmp->next;
        }
    }
    delete tmp;
    return true;
}

void processNext()
{
    if (current == NULL)
    {
        cout << "No active requests to process." << endl;
        return;
    }

    node *temp = current;
    int slice;
    if (temp->RemainingTime < QUANTUM)
    {
        slice = temp->RemainingTime;
    }
    else
    {
```

```
        slice = QUANTUM;
    }

    temp->RemainingTime = temp->RemainingTime - slice;
    cout<<"Processing Request "<<temp->RequestID<<" ( User "<<temp->UserID<<",
"<<temp->ModelName
        <<" ) for "<<slice<<" units. Remaining : "<<temp->RemainingTime<<endl;

    if (temp->RemainingTime == 0)
    {
        cout<<"Request "<<temp->RequestID<<" completed and removed."<<endl;
        DeleteRequest(temp->RequestID);
    }
    else
    {
        current = temp->next;
    }
}

void showAllRequests()
{
    if (last == NULL)
    {
        cout << "No active requests!" << endl;
        return;
    }

    cout<<"---Active Requests---"<<endl;
    node *temp = last->next;
    do{
        cout<<"Req "<<temp->RequestID<<" | User "<<temp->UserID<<" | "<<temp-
>ModelName
            <<" | Remaining : "<<temp->RemainingTime;
        if (temp == current)
        {
            cout<<"  <-- next turn";
        }
        cout<<endl;
        temp = temp->next;
    }while(temp != last->next);
}

void SearchRequest(int value)
{
    if (last == NULL)
    {
        cout << "No active requests!" << endl;
        return;
    }

    node *temp = last->next;
    int pos = 1;

    do{
        if(temp->RequestID == value)
        {
```

```
            cout<<value<< " Exist at Positon : "<< pos<<endl;
            cout<<"User "<<temp->UserID<<endl;
            cout<<temp->ModelName<<endl;
            cout<<"Remaining : "<<temp->RemainingTime<<endl;
            return;
        }
        temp = temp->next;
        pos++;
    }while(temp != last->next);

    cout<<value<<" does not exist. "<<endl;
}

void simulate10()
{
    for(int i=1; i<=10; i++)
    {
        cout<<"Cycle "<<i<<" : ";
        if (current == NULL)
        {
            cout<<"queue empty, scheduler idle."<<endl;
            continue;
        }
        processNext();
    }
}


int main()
{
    int x,value,uid,time;
    string model;

    do{
    cout << " AI Request Scheduler ( Round Robin, Quantum = " << QUANTUM << " ) " <<
endl;
    cout << "1.Add a new request at the end." <<endl;
    cout << "2.Remove a request using Request ID." <<endl;
    cout << "3.Process the next request."<<endl;
    cout << "4.Display all active requests."<<endl;
    cout << "5.Find a request."<<endl;
    cout << "6.Simulate 10 scheduling cycles." <<endl;
    cout << "0. Exit "<<endl;
    cin >> x;
    cout<<endl;

    if (cin.fail())
    {
        cin.clear();
        cin.ignore(1000, '\n');
        x = -1;
    }

    if(x<0 || x>6)
    {
        cout<< "Invalid input! TRY AGAIN "<<endl;
```

```
    }
    if (x == 1) {
            cout << "Enter RequestID: ";
            cin >> value;
            cout << "Enter UserID: ";
            cin >> uid;
            cout << "Enter Model Name: ";
            cin >> model;
            cout << "Enter Processing Time: ";
            cin >> time;
            addRequest(value, uid, model, time);
        }
        else if (x == 2) {
            cout << "Enter RequestID which you want to remove: ";
            cin >> value;
            if (DeleteRequest(value) == true)
            {
                cout<<"Request "<<value<<" removed."<<endl;
            }
        }
        else if (x == 3) {
            processNext();
        }
        else if (x == 4) {
            showAllRequests();
        }
        else if (x == 5) {
            cout << "Enter RequestID you want to Search: ";
            cin >> value;
            SearchRequest(value);
        }
        else if (x == 6) {
            simulate10();
        }
    } while (x != 0);

    if (last != NULL)
    {
        node *tem = last->next;
        last->next = NULL;
        while(tem != NULL)
        {
            node *Delet = tem;
            tem = tem->next;
            delete Delet;
        }
    }

    cout << "Program Ends! Thank you\n";
    return 0;
}
```

Screenshot:	1.Add a new request at the end

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
1

Enter RequestID: 001
Enter UserID: 1
Enter Model Name: claude
Enter Processing Time: 2
Request 1 added ( User 1, claude, 2 units ).
```

2.Remove a request using Request ID

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
2

Enter RequestID which you want to remove: 1
Request 1 removed.
```

3.Process the next request

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
3

No active requests to process.
```

4.Display all active requests

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
4

No active requests!
```

5.Find a request

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
5

Enter RequestID you want to Search: 1
No active requests!
```

6.Simulate 10 scheduling cycles

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
6

Cycle 1 : queue empty, scheduler idle.
Cycle 2 : queue empty, scheduler idle.
Cycle 3 : queue empty, scheduler idle.
Cycle 4 : queue empty, scheduler idle.
Cycle 5 : queue empty, scheduler idle.
Cycle 6 : queue empty, scheduler idle.
Cycle 7 : queue empty, scheduler idle.
Cycle 8 : queue empty, scheduler idle.
Cycle 9 : queue empty, scheduler idle.
Cycle 10 : queue empty, scheduler idle.
```

0.Exit

```
 AI Request Scheduler ( Round Robin, Quantum = 3 )
1.Add a new request at the end.
2.Remove a request using Request ID.
3.Process the next request.
4.Display all active requests.
5.Find a request.
6.Simulate 10 scheduling cycles.
0. Exit
0

Program Ends! Thank you
```